

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский
Томский политехнический университет

Инженерная школа информационных технологий и робототехники
Отделение информационных технологий

Отчет по лабораторной работе №24 по дисциплине

«Язык Kotlin и основы разработки»

Рисование

Выполнил:

Студент группы 1А22



О.К. Кравцов

Проверил:

Ст. преп. ОИТ ИШИТР



В.А. Дорофеев

Задание

Придумайте и реализуйте приложение, в котором используется рисование. На экране должно быть несколько движущихся или изменяющих свои визуальные свойства объектов: например, машинки ездят по экрану, солнце перемещается по дуге (и, возможно, сменяется луной), мыльные пузыри летают по экрану и сталкиваются друг с другом, звезды на небе случайно (но плавно) мерцают, ... Удивите преподавателя (в приятном смысле!)

Ход работы

1. Создан проект Lab24 на основе Empty Views Activity.
2. Настроены ресурсы приложения:

– colors.xml

```
<resources>
    <color name="background">#FF000000</color>
    <color name="lightning_white">#FFFFFF</color>
</resources>
```

– strings.xml

```
<resources>
    <string name="app_name">Lab24</string>
</resources>
```

– dims.xml

```
<resources>
    <dimen name="lightning_stroke_width">1dp</dimen>
</resources>
```

– integers.xml

```
<resources>
    <integer name="animation_delay">16</integer>
</resources>
```

3. Основной интерфейс приложения состоит из одного элемента PlasmaView, занимающего весь экран:

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layerType="hardware"
    tools:context=".MainActivity">

    <ru.olegkravtsov.lab24.PlasmaView
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</FrameLayout>
```

4. Создана основная активность приложения, которая выполняет роль контейнера для пользовательского интерфейса. Основная задача MainActivity — инициализировать макет activity_main и обеспечить корректный жизненный цикл для отображения анимации. Код активности минимален и состоит только из метода onCreate(), который устанавливает макет с помощью setContentView():

```
package ru.olegkravtsov.lab24

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

5. Создан кастомный класс PlasmaView, наследующий от View, который реализует анимированное плазменное поле с динамически меняющимися цветами и генерацию случайных молний с плавным затуханием. Для оптимизации производительности использованы предварительные вычисления в методе precomputeStaticData().

```
package ru.olegkravtsov.lab24

import android.content.Context
import android.graphics.*
import android.util.AttributeSet
import android.view.View
import androidx.core.content.ContextCompat
import kotlin.math.*
import kotlin.random.Random

class PlasmaView @JvmOverloads constructor(
    context: Context,
    attrs: AttributeSet? = null,
    defStyleAttr: Int = 0
) : View(context, attrs, defStyleAttr) {

    private val lightningPaint: Paint = Paint().apply {
        isAntiAlias = true
        style = Paint.Style.STROKE
        strokeWidth = resources.getDimension(R.dimen.lightning_stroke_width)
        color = getColorFromResource(R.color.lightning_white)
        strokeCap = Paint.Cap.ROUND
    }

    private var time: Float = 0f
    private val animator: Runnable = object : Runnable {
        override fun run() {
            updateAnimation()
            invalidate()
            handler.postDelayed(this,
resources.getInteger(R.integer.animation_delay).toLong())
        }
    }

    private lateinit var plasmaBitmap: Bitmap
    private val bitmapPaint: Paint = Paint().apply {
        isFilterBitmap = true
        isDither = true
    }

    private val plasmaArray = IntArray(128 * 128)
    private val precomputedValues = FloatArray(128 * 128)

    private data class LightningBolt(
        var life: Float = 1f,
        var segments: List<LineSegment> = listOf()
    )

    private data class LineSegment(
        val startX: Float, val startY: Float,
        val endX: Float, val endY: Float
    )

    private val lightningBolts = mutableListOf<LightningBolt>()
```

```

private fun getColorFromResource(colorResId: Int): Int {
    return ContextCompat.getColor(context, colorResId)
}

override fun onAttachedToWindow() {
    super.onAttachedToWindow()
    plasmaBitmap = Bitmap.createBitmap(128, 128, Bitmap.Config.RGB_565)
    precomputeStaticData()
    startAnimation()
}

private fun precomputeStaticData() {
    for (y in 0 until 128) {
        for (x in 0 until 128) {
            val dx = x - 64f
            val dy = y - 64f
            val distance = sqrt(dx * dx + dy * dy)

            val index = y * 128 + x
            precomputedValues[index] = (
                sin(x * 0.1f) +
                sin(y * 0.1f) +
                sin((x + y) * 0.15f) +
                sin(distance * 0.2f)
            )
        }
    }
}

private fun updateAnimation() {
    time += 0.08f
    generatePlasma()
    updateLightning()

    if (Random.nextFloat() < 0.03f) {
        createLightning()
    }
}

private fun updateLightning() {
    lightningBolts.removeAll { bolt ->
        bolt.life -= 0.008f
        bolt.life <= 0f
    }
}

private fun createLightning() {
    createVerticalLightning()
}

private fun createVerticalLightning() {
    val startX = Random.nextFloat() * width
    val startY = 0f
    val endX = startX + (Random.nextFloat() - 0.5f) * width * 0.2f
    val endY = height.toFloat()

    val segments = generateSimpleLightning(startX, endX, endY)
    val bolt = LightningBolt(life = 1.2f, segments = segments)
    lightningBolts.add(bolt)
}

private fun generateSimpleLightning(x1: Float, x2: Float, y2: Float):
List<LineSegment> {
    val result = mutableListOf<LineSegment>()

```

```

        val points = mutableListOf<PointF>()

        points.add(PointF(x1, 0f))
        points.add(PointF(x2, y2))

        for (i in 1 until 3) {
            val ratio = i.toFloat() / 3
            val baseX = x1 + (x2 - x1) * ratio
            val baseY = y2 * ratio

            val offsetX = (Random.nextFloat() - 0.5f) * (width * 0.15f)
            val offsetY = (Random.nextFloat() - 0.5f) * (height * 0.1f)

            points.add(i, PointF(baseX + offsetX, baseY + offsetY))
        }

        for (i in 0 until points.size - 1) {
            result.add(LineSegment(
                points[i].x, points[i].y,
                points[i + 1].x, points[i + 1].y
            ))
        }

        return result
    }

    private fun generatePlasma() {
        if (!::plasmaBitmap.isInitialized) return

        for (y in 0 until 128) {
            for (x in 0 until 128) {
                val index = y * 128 + x
                val value = (precomputedValues[index] + time * 2) * 0.5f + 0.5f

                val color = getPlasmaColor(value)
                plasmaArray[index] = color
            }
        }

        plasmaBitmap.setPixels(plasmaArray, 0, 128, 0, 0, 128, 128)
    }

    private fun getPlasmaColor(value: Float): Int {
        val r = (sin(value * PI * 2 + time) * 127 + 128).toInt()
        val g = (sin(value * PI * 2 + 2.094f + time * 0.7f) * 127 + 128).toInt()
        val b = (sin(value * PI * 2 + 4.188f + time * 1.3f) * 127 + 128).toInt()

        return Color.rgb(
            r.coerceIn(0, 255),
            g.coerceIn(0, 255),
            b.coerceIn(0, 255)
        )
    }

    override fun onDraw(canvas: Canvas) {
        canvas.drawColor(getColorFromResource(R.color.background))

        if (::plasmaBitmap.isInitialized) {
            val srcRect = Rect(0, 0, 128, 128)
            val dstRect = Rect(0, 0, width, height)
            canvas.drawBitmap(plasmaBitmap, srcRect, dstRect, bitmapPaint)
        }

        drawLightning(canvas)
    }

```

```

    }

    private fun drawLightning(canvas: Canvas) {
        lightningBolts.forEach { bolt ->
            val alpha = (bolt.life * 255).toInt()
            lightningPaint.alpha = alpha

            bolt.segments.forEach { segment ->
                canvas.drawLine(segment.startX, segment.startY, segment.endX,
segment.endY, lightningPaint)
            }
        }
    }

    override fun onSizeChanged(w: Int, h: Int, oldw: Int, oldh: Int) {
        super.onSizeChanged(w, h, oldw, oldh)
        if (w > 0 && h > 0 && ::plasmaBitmap.isInitialized) {
            plasmaBitmap.recycle()
            plasmaBitmap = Bitmap.createBitmap(128, 128, Bitmap.Config.RGB_565)
        }
    }

    private fun startAnimation() {
        handler.removeCallbacks(animatort)
        handler.post(animatort)
    }

    override fun onDetachedFromWindow() {
        super.onDetachedFromWindow()
        handler.removeCallbacks(animatort)
        if (::plasmaBitmap.isInitialized) {
            plasmaBitmap.recycle()
        }
    }
}

```

Результат работы

Разработанное приложение демонстрирует динамическую визуализацию плазмы с эффектами молний, эффект похож на разбитый экран с трещинами (рис. 1). При запуске пользователь видит полноэкранную анимацию, где на фоне непрерывно меняются цвета плазмы, а на переднем плане появляются и исчезают тонкие ломанные линии. Основу визуализации составляет плазменный эффект, где цветовые переходы основаны на тригонометрических функциях, что создаёт плавные волновые эффекты. Цвета циклически меняются со временем, обеспечивая бесконечную анимацию.

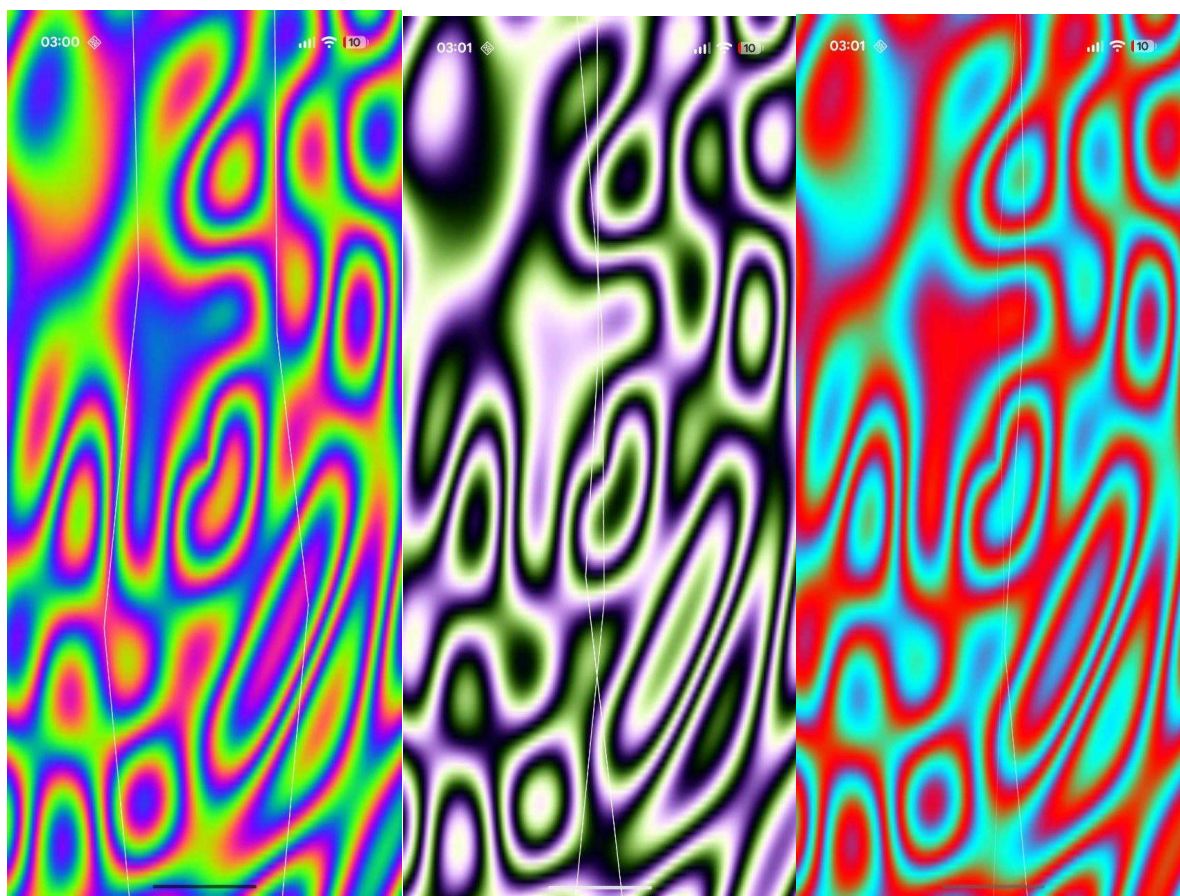


Рисунок 1 – Полученный визуальный эффект

Выводы

В ходе лабораторной работы было успешно разработано приложение, использующее возможности кастомного рисования в Android. Были освоены принципы работы с классами Canvas, Paint и Bitmap для создания сложных графических эффектов, что позволило реализовать динамическую анимацию с использованием отдельного потока обновления. Создан сложный визуальный эффект плазмы с применением тригонометрических функций для генерации цветов, а также дополнительный эффект молний с управлением их жизненным циклом. В результате оптимизации использования памяти и производительности через рециклинг ресурсов и предварительные вычисления было достигнуто плавное выполнение анимации.